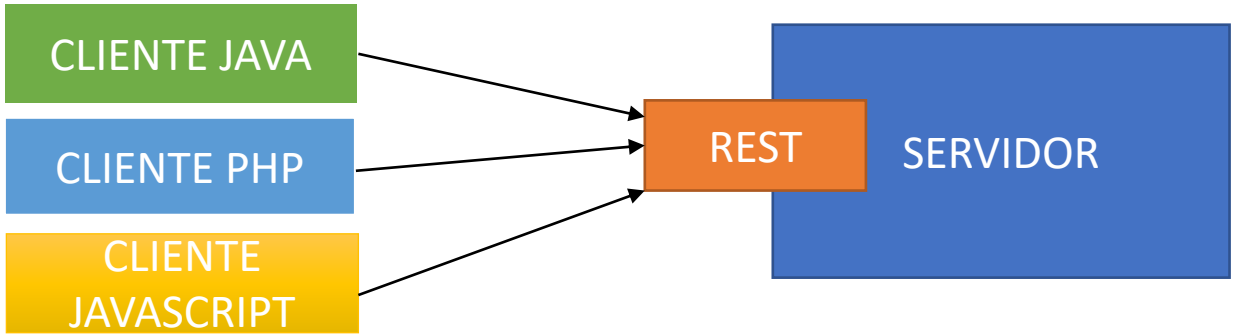
Servicios REST

- Significa ***RE**presentational **S**tate **T**ransfer*
- Cuando trabajamos con servicios REST estamos trabajando con una arquitectura Cliente Servidor, en la cual el servidor publica la información en formato de dato puro.
- Normalmente, los datos están en formato JSON(**J**ava**S**cript **O**bject **N**otation)
- Los datos publicados por el servidor, pueden ser accedidos por una aplicación cliente que se encargará de procesarlos y presentar el resultado en una aplicación de usuario.

Servicios REST

- **REST** presenta un conjunto de restricciones que sigue los siguientes principios:
  - **Stateless**: Los servicios REST no se encargan de mantener **ningún tipo de estado** entre peticiones; cada una de las peticiones es totalmente independiente de la siguiente. Al no mantener estado nos encontramos que **se incrementa la escalabilidad** de estos.
  - **Neutralidad tecnológica**: Permite a prácticamente cualquier tipo de cliente y de lenguaje conectarse servicios REST:



Servicios REST

- **URLs orientadas a recursos**: Cada una de ellas gestiona todas las operaciones que un recurso concreto soporta . Ejemplos de recursos pueden ser /pedidos /productos /clientes… Una posible lista de URIs que expondrá nuestro sistema podría ser:
  - /pedidos
  - /pedidos/{id}
  - /productos
  - /productos/{id}
  - /clientes
  - /clientes/{id}

Servicios REST

- **Hace uso de los métodos clásicos** del protocolo HTTP:
  - GET: Solicita información al servicio
  - POST : Inserta información del servicio
  - PUT : Actualiza la información del servicio
  - PATH: Actualiza parcialmente la información del servicio
  - DELETE : Borra la información del servicio

- **Uniformidad**

Acción sobre los datos	Protocolo HTTP equivalente
CREATE	POST
RETRIEVE	GET
UPDATE	PUT
DELETE	DELETE

Servicios REST

Códigos correctos

Acción	Código	Descripción
GET	200	OK
POST	201	Creado
PUT	204	Sin contenido
DELETE	204	Sin contenido

Códigos error cliente

Código	Descripción
400	Bad Requet
401	Unauthoried
403	Forbidden
404	Not found
409	Conflict

Códigos error servidor

Código	Descripción
500	Internal Server Error
501	Not implemented
503	Service unavailable

Servicios REST con SpringBoot

- Controladores REST con @RestController

```
@RestController
public class GreetingController {
    @GetMapping("/greeting")
    public Greeting greeting(@RequestParam Optional<String> name) {
        return new Greeting(counter.incrementAndGet(),
                            String.format(template, name));
    }
}
```

Servicios REST con SpringBoot

- Mapeo de métodos
  - @GetMapping
  - @PostMapping
  - @PutMapping
  - @DeleteMapping
- Lectura de variables de la petición
  - @RequestParam
  - @PathVariable
  - @RequestBody

Servicios REST con SpringBoot

Para trabajar con las respuestas que devolverá el trabajar, se puede utilizar la clase:

- ResponseEntity<T>
  - Nos permite manejar de una forma más conveniente la respuesta que enviamos al cliente.
  - Hereda de HttpEntity<T>
  - Nos permite indicar el código de respuesta, qué se envía en el cuerpo, responder peticiones sin el mismo

- Tutorial REST con Spring-Boot
  - https://spring.io/guides/tutorials/rest/

Servicios REST con SpringBoot. Clientes

- Podemos consumir de un API REST con Spring de forma sencilla:
  - Clase RestTemplate

```
RestTemplate restTemplate = new RestTemplate();
HttpEntity<Foo> request = new HttpEntity<>(new Foo("bar"));
ResponseEntity<Foo> response = restTemplate
  .exchange(fooResourceUrl, HttpMethod.POST, request, Foo.class);
```